

PROJECT REPORT

Online Donation and Charity Management System

A Full-Stack Web Application built with ASP.NET Core, React, and MySQL

Technology Stack

Backend: ASP.NET Core Web API | Frontend: React.js | Database: MySQL

1. Introduction

Charitable organizations rely on public trust and financial transparency to sustain their operations. Traditional, manual methods of collecting donations and managing donor records are slow, error-prone, and difficult to audit. This project proposes a web-based Online Donation and Charity Management System that digitizes the entire donation lifecycle - from donor registration and secure payment collection to campaign tracking and financial reporting - using a modern three-tier architecture built on ASP.NET Core, React, and MySQL.

This document explains, in plain and practical terms, how to plan, design, and build the system step by step, so that a student or development team can follow it as an execution roadmap.

2. Problem Statement

Nonprofit organizations that rely on manual or semi-manual processes typically face the following challenges:

- Donation tracking issues - donations recorded on paper or in spreadsheets are easily lost, duplicated, or misreported.
- Lack of financial transparency - donors and auditors cannot easily verify how funds are collected and spent.
- Difficulty managing donor records - no centralized database of donor history, preferences, or communication.
- Delayed fund allocation - manual approval chains slow down the release of funds to active campaigns or beneficiaries.
- Poor campaign monitoring - organizations struggle to track real-time progress of fundraising goals across multiple campaigns.

3. Objectives

The system is designed to achieve the following objectives:

- Automate the end-to-end donation process, from donor registration to payment confirmation and receipt generation.
- Maintain a centralized, secure donor database with complete donation history.
- Enable creation and real-time monitoring of fundraising campaigns with progress indicators.
- Provide transparent financial reports for administrators, auditors, and donors.
- Support role-based access for Admin, Campaign Manager, and Donor user types.
- Ensure secure, PCI-aware handling of online payments through a trusted payment gateway.

4. Scope of the Project

The system scope covers the following functional areas:

- Donor registration, login, and profile management.
- One-time and recurring donation processing through integrated payment gateways.
- Campaign creation, categorization (e.g., education, medical, disaster relief), and progress tracking.
- Automatic receipt and acknowledgement generation (email/PDF).
- Admin dashboard for fund allocation, campaign approval, and donor management.

- Financial reporting: donation summaries, campaign-wise reports, and exportable statements.
- Notifications for donation confirmation, campaign milestones, and tax-receipt issuance.

5. System Architecture

The system follows a three-tier architecture that cleanly separates the presentation layer, the business/API layer, and the data layer. This separation makes the system easier to scale, secure, and maintain.

5.1 Architecture Layers

Layer	Technology	Responsibility
Presentation (Client)	React.js, React Router, Axios/Fetch, Bootstrap/Tailwind	Renders UI, handles user interaction, calls REST APIs
Application (Server)	ASP.NET Core Web API, C#, Entity Framework Core	Business logic, authentication, validation, REST endpoints
Data	MySQL, Stored Procedures/Views	Persistent storage of donors, donations, campaigns, transactions
External Services	Payment Gateway (Razorpay/Stripe/PayPal), SMTP/Email API	Payment processing and email/SMS notifications

5.2 Request Flow

A typical donation request flows through the system as follows:

1. The donor interacts with the React frontend and submits a donation form.
2. React sends an HTTPS request (via Axios) to the ASP.NET Core Web API.
3. The API validates the request, applies business rules, and communicates with the payment gateway.
4. On successful payment, the API writes the transaction to the MySQL database using Entity Framework Core.
5. A confirmation response is returned to React, and an email/PDF receipt is triggered asynchronously.
6. Campaign totals and dashboards are updated in real time (or on next refresh) to reflect the new donation.

6. Technology Stack

Component	Technology / Tool
Frontend	React.js, React Router DOM, Axios, Redux Toolkit or Context API, Bootstrap / Tailwind CSS
Backend	ASP.NET Core Web API (C#), Entity Framework Core, AutoMapper
Database	MySQL 8.x, MySQL Workbench (design), Pomelo.EntityFrameworkCore.MySql (connector)
Authentication	ASP.NET Core Identity + JWT (JSON Web Tokens)
Payment Gateway	Razorpay / Stripe / PayPal sandbox API

Component	Technology / Tool
Reporting	SQL Views/Stored Procedures, QuestPDF or Rotativa for PDF export, Chart.js/Recharts for dashboards
Notifications	SMTP (SendGrid/Gmail API) for email receipts
Version Control & CI/CD	Git, GitHub, GitHub Actions
Hosting	IIS / Azure App Service (API), Netlify/Vercel or Azure Static Web Apps (React), Azure Database for MySQL / on-prem MySQL server

7. Database Design

The MySQL schema is normalized to third normal form (3NF) to avoid data redundancy. The core tables are described below.

7.1 Users Table (Donors, Admins, Managers)

Column	Type	Description
UserId	INT (PK, AUTO_INCREMENT)	Unique identifier
FullName	VARCHAR(100)	Name of the user
Email	VARCHAR(100), UNIQUE	Login email
PasswordHash	VARCHAR(255)	Hashed password (never plain text)
Role	ENUM('Donor','Admin','Manager')	Role-based access control
Phone	VARCHAR(15)	Contact number
CreatedAt	DATETIME	Account creation timestamp

7.2 Campaigns Table

Column	Type	Description
CampaignId	INT (PK, AUTO_INCREMENT)	Unique identifier
Title	VARCHAR(150)	Campaign name
Description	TEXT	Campaign details
Category	VARCHAR(50)	e.g. Education, Medical, Disaster Relief
GoalAmount	DECIMAL(12,2)	Target amount
RaisedAmount	DECIMAL(12,2)	Amount raised so far
StartDate / EndDate	DATE	Campaign duration
Status	ENUM('Active','Completed','Suspended')	Current campaign state
CreatedBy	INT (FK -> UserId)	Manager/Admin who created it

7.3 Donations Table

Column	Type	Description
DonationId	INT (PK, AUTO_INCREMENT)	Unique identifier
DonorId	INT (FK -> UserId)	Donor who made the donation
CampaignId	INT (FK -> CampaignId)	Campaign receiving the funds
Amount	DECIMAL(10,2)	Donation amount
PaymentMode	VARCHAR(30)	Card/UPI/Net-banking/Wallet
TransactionId	VARCHAR(100)	Payment gateway reference
Status	ENUM('Pending','Success','Failed','Refunded')	Payment status
DonationDate	DATETIME	Timestamp of donation
IsRecurring	BOOLEAN	Whether donation is a recurring pledge

7.4 FundAllocations and Reports Tables

Column	Type	Description
AllocationId	INT (PK, AUTO_INCREMENT)	Unique identifier
CampaignId	INT (FK)	Campaign the funds are allocated to
AllocatedAmount	DECIMAL(12,2)	Amount released for use
Purpose	VARCHAR(200)	Reason for allocation (e.g., medical supplies)
ApprovedBy	INT (FK -> UserId)	Admin who approved the allocation
AllocationDate	DATETIME	Date of fund release

Additional supporting tables include Notifications, Receipts, and AuditLogs, which record system emails, generated PDF receipts, and administrative actions respectively, further strengthening transparency.

8. Core Modules

8.1 Donor Module

- Registration/login with email verification and JWT-based authentication.
- Browse active campaigns and make one-time or recurring donations.
- View personal donation history and download tax receipts.

8.2 Campaign Management Module

- Create, edit, and categorize campaigns with goal amounts and deadlines.
- Track real-time progress bars showing raised vs. goal amount.
- Close or suspend campaigns automatically when the goal or end date is reached.

8.3 Payment Processing Module

- Integrate a secure payment gateway (e.g., Razorpay/Stripe) using server-side order creation and client-side checkout.

- Verify payment signatures on the backend before marking a donation as successful.
- Handle failed/refunded transactions gracefully with proper status updates.

8.4 Admin & Fund Allocation Module

- Approve campaigns submitted by managers before they go live.
- Allocate raised funds to specific purposes and record justifications.
- Manage user roles and monitor overall platform activity.

8.5 Reporting & Transparency Module

- Generate donation summary reports by date range, campaign, or donor.
- Export financial statements as PDF/Excel for auditing.
- Visual dashboards (charts) showing fund inflow, allocation, and campaign performance.

9. Step-by-Step Implementation Plan

The project can be executed in the following phased steps. Each phase builds on the previous one and can be treated as a milestone/sprint.

Step 1: Requirement Analysis & Planning

- Gather functional requirements from stakeholders (NGO staff, donors, auditors).
- Define user roles: Donor, Campaign Manager, Admin.
- Prepare use-case diagrams, ER diagrams, and a project timeline.

Step 2: Database Design (MySQL)

- Design the ER diagram covering Users, Campaigns, Donations, Allocations, and Reports.
- Create the MySQL schema using MySQL Workbench; define primary/foreign keys and constraints.
- Write seed scripts for test data (sample donors and campaigns).

Step 3: Backend Setup (ASP.NET Core Web API)

- Create a new ASP.NET Core Web API project (dotnet new webapi).
- Configure Entity Framework Core with the Pomelo MySQL provider and connect to the MySQL database.
- Scaffold models (Donor, Campaign, Donation, Allocation) and generate migrations (dotnet ef migrations add Initial).
- Implement the repository/service layer for business logic, separate from controllers.
- Set up ASP.NET Core Identity with JWT authentication and role-based authorization.

Step 4: REST API Development

- Build controllers/endpoints: /api/auth, /api/donors, /api/campaigns, /api/donations, /api/reports.
- Add model validation (Data Annotations/FluentValidation) and centralized error handling middleware.
- Document APIs with Swagger/OpenAPI for easy testing and frontend integration.

Step 5: Payment Gateway Integration

- Register a merchant account with a payment gateway (sandbox/test mode first).
- Implement server-side order creation and signature verification endpoints.

- Update donation status only after verifying the payment callback/webhook.

Step 6: Frontend Development (React)

- Scaffold the React app (Vite or Create React App) and set up React Router for navigation.
- Build reusable components: Navbar, Campaign Card, Donation Form, Dashboard widgets.
- Connect to backend APIs using Axios; manage auth state and tokens (Context API or Redux Toolkit).
- Build donor pages (browse campaigns, donate, view history) and admin pages (approve campaigns, view reports).

Step 7: Reporting & Dashboard

- Create SQL views/stored procedures to aggregate donation and campaign data.
- Build chart-based dashboards (Chart.js/Recharts) for visual fund tracking.
- Add PDF/Excel export for financial statements using a reporting library (e.g., QuestPDF).

Step 8: Security Hardening

- Hash passwords with a strong algorithm (e.g., bcrypt via ASP.NET Core Identity).
- Enforce HTTPS, CORS policies, and input sanitization to prevent SQL injection/XSS.
- Use JWT expiration/refresh tokens and role checks on every sensitive endpoint.

Step 9: Testing

- Unit test backend services and controllers (xUnit/NUnit).
- Test API endpoints with Postman/Swagger, covering both valid and invalid inputs.
- Perform UI testing of the React app and end-to-end testing of the full donation flow.

Step 10: Deployment

- Deploy the ASP.NET Core API to IIS or Azure App Service.
- Deploy the React build to Netlify/Vercel or Azure Static Web Apps.
- Host the MySQL database on Azure Database for MySQL or a managed MySQL server; configure backups.
- Set up a CI/CD pipeline (GitHub Actions) to automate build, test, and deployment.

Step 11: Maintenance & Monitoring

- Monitor application logs and error rates (e.g., Application Insights or Serilog).
- Collect user feedback and iterate on campaign/reporting features.
- Periodically audit financial data for transparency and compliance.

10. Security Considerations

- Role-based access control (RBAC) separating Donor, Manager, and Admin permissions.
- All payment data handled through the gateway's secure APIs; no raw card data stored locally.
- HTTPS enforced across frontend and backend communication.
- Audit logs for all fund allocation and administrative actions to preserve transparency.
- Regular database backups and encrypted storage of sensitive fields (e.g., PII).

11. Testing Strategy

Test Type	Tools	Focus Area
Unit Testing	xUnit / NUnit	Business logic, services, calculations
API Testing	Postman, Swagger	Endpoint correctness, status codes, auth
Integration Testing	xUnit + In-Memory/Test DB	API to database interaction
UI Testing	React Testing Library, Jest	Component rendering and user flows
End-to-End Testing	Cypress/Playwright	Full donation-to-receipt journey
Security Testing	OWASP ZAP	SQL injection, XSS, auth bypass checks

12. Future Enhancements

- Mobile app (React Native) for donors to give on the go.
- Blockchain-based ledger for immutable donation transparency.
- AI-driven campaign recommendations based on donor interest history.
- Multi-currency and multi-language support for global donors.
- Integration with government tax-exemption portals for automated tax receipt filing.

13. Conclusion

The Online Donation and Charity Management System addresses the core pain points of manual charity operations - poor tracking, weak transparency, and slow fund allocation - by combining a React frontend, an ASP.NET Core Web API backend, and a MySQL database into a secure, scalable platform. Following the phased implementation plan above, a development team can progress methodically from requirement analysis and database design through to deployment and ongoing maintenance, delivering a system that builds donor trust through automation, accuracy, and transparency.